FinSight

Regulatory-Grade Financial Intelligence System

Project Design & Engineering Specification

Version 1.0 | 6-Week Build Plan | Production-Grade

Component	Technology	Rationale
LLM Inference	Groq (Llama 3.3 70B)	Free tier, fastest open inference
Orchestration	Semantic Kernel (Python)	SK planner is load-bearing
Vector Store	Qdrant (Docker)	Production-grade, local, free
Embeddings	sentence-transformers	Local, no API dependency
Reranker	ms-marco cross-encoder	Better relevance than vector score
API	FastAPI + async Python	Clean engineering signal
PDF Parsing	PyMuPDF	Best positional extraction
Observability	OpenTelemetry + structlog	Full trace per agent hop
Containerisation	Docker + docker-compose	One-command reproducibility

One-Line Project Summary

A multi-agent financial analysis system that takes arbitrary natural-language analysis tasks, decomposes them autonomously via SK's Stepwise Planner, retrieves evidence from real financial filings, and produces structured reports where every claim is traceable to a verbatim source passage — with uncertain claims flagged and every run producing a machine-readable audit log.

1. Project Goals

1.1 Why This Project

The standard GenAI portfolio project in 2026 is a RAG chatbot over PDFs. Every bootcamp graduate has one. The problem is not that RAG is wrong — it is that most implementations treat reliability as an afterthought: hallucinations are not caught, citations are cosmetic, and there is no audit trail. In compliance-heavy domains like financial services and consulting, an unverifiable AI output is not just unhelpful — it is a liability.

FinSight is designed from first principles around the question: what would a GenAI system need to look like for a compliance team to actually trust it? The answer drives every architectural decision.

1.2 Primary Goals

- Learn Semantic Kernel deeply — not as a wrapper but as a framework whose design philosophy you can articulate and defend
- Build a production-grade system that demonstrates senior engineering judgment: every decision has a reason, every failure mode has a handler
- Create a portfolio piece that reads as immediately credible to a financial services or consulting engineering team
- Develop genuine understanding of the hard problems in production RAG: retrieval quality, hallucination mitigation, confidence modeling, observability

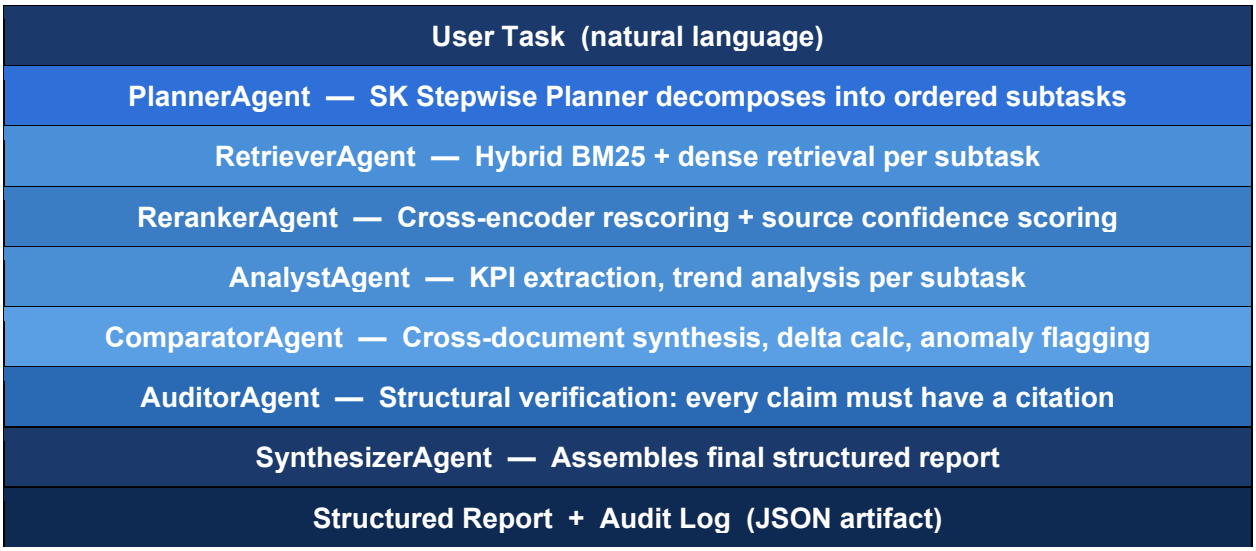
1.3 What Makes This Non-Trivial

Signal	What It Demonstrates
SK Stepwise Planner is load-bearing	Task decomposition is dynamic, not hardcoded. The plan changes with the query.
Citation is structural, not cosmetic	Citations thread through every agent hop. The pipeline cannot produce output without them.
Domain-specific confidence model	Section type awareness — audited financials vs MD&A commentary — is not in generic RAG systems.
Cross-document reasoning	ComparatorAgent handles multi-source claims with multi-source citations. Most RAG systems do not.
AuditorAgent = compliance layer	A dedicated verification pass that rejects unverifiable claims structurally, not via prompt instruction.
Eval framework you own	Adversarial queries, retrieval recall, hallucination rate. Not a third-party black box.

2. System Architecture

2.1 High-Level Pipeline

A user submits a natural-language analysis task. The system decomposes it, retrieves evidence, executes specialised agents, verifies every claim, and produces a structured report with a machine-readable audit log.



2.2 Agent Responsibilities

Agent	Input	Output	SK Pattern
PlannerAgent	User task string	Ordered list of subtasks	Stepwise Planner
RetrieverAgent	Subtask string	Ranked chunks with citations	Native plugin
AnalystAgent	Chunks + citations	KPIs, figures, trends	Native plugin
ComparatorAgent	Multi-doc analysis results	Deltas, anomalies, cross-citations	Native plugin
AuditorAgent	All claims + citations	Verified / Uncertain / Unverifiable	Native plugin
SynthesizerAgent	Verified claims	Structured report JSON	Semantic function

2.3 The Citation Object

Every claim in the system carries this object through every agent hop. Citations are never added at the end — they are structural.

```
{
  "document":      "Infosys_AR_2024.pdf",
  "page":           31,
  "snippet":        "Operating profit margin for FY2024 stood at 20.7%,
                    compared to 21.5% in FY2023.",
  "claim":          "Infosys operating margin declined 0.8pp in FY2024",
  "confidence":     0.91,
  "section_type":   "audited_financials"
}
```

Why snippet vs claim is critical

The distinction between snippet (verbatim source text) and claim (what the agent asserted) makes hallucination visible. If the claim does not follow from the snippet, a human reviewer catches it instantly. This is not a prompt-level safeguard — it is structural.

2.4 Source Confidence Model

Each retrieved chunk receives a composite confidence score from five signals. This is the domain-specific contribution that generic RAG systems lack.

Signal	What It Captures	High Score	Low Score
Retrieval score	Relevance to query after reranking	Direct answer to query	Tangential passage
Section type	Where in the filing the chunk appears	Audited financials, notes	MD&A commentary, letters
Freshness	Recency of the source filing	Most recent fiscal year	3+ year old filing
Cross-filing consistency	Does figure appear in multiple filings	Same figure in 3 filings	Appears once only
Retrieval consistency	Does chunk surface across query phrasings	Surfaces for 3+ variants	Only one phrasing retrieves it

If composite confidence falls below threshold, the AuditorAgent does not flag the claim as uncertain — it triggers the hallucination fallback: insufficient evidence found. The threshold is configurable and documented in DECISIONS.md.

2.5 The AuditorAgent — The Differentiator

A dedicated verification pass before any output leaves the system. This is what makes FinSight compliance-native rather than compliance-adjacent.

Audit Status	Condition	Action
VERIFIED	Claim has citation; snippet supports claim; confidence above threshold	Included in report unmarked
UNCERTAIN	Claim has citation; confidence is low; or snippet is weak support	Included with explicit flag and confidence score
UNVERIFIABLE	No citation found; or claim contradicts retrieved context	Blocked from report entirely; logged in audit trail

Why this is structural, not prompt-level

Prompt instructions like 'only answer from context' can be overridden by sufficiently strong model priors or adversarial inputs. The AuditorAgent is a separate agent that performs entailment checking: 'does this snippet support this claim, yes or no, explain.' Claims that fail are blocked before synthesis. You cannot prompt-engineer your way around a structural verification pass.

2.6 The Audit Log

Every analysis run produces a JSON artifact alongside the report. This is the compliance paper trail — every analysis is reproducible and explainable.

```
{
  "task_id":              "uuid-v4",
  "timestamp":            "2024-03-15T10:23:00Z",
  "user_query":           "Compare Infosys and TCS margins FY2022-2024",
  "plan":                 ["retrieve_infosys_margins", "retrieve_tcs_margins",
"compare_deltas"],
  "retrievals":           { "retrieve_infosys_margins": ["chunk_ids + scores"]
},
  "claims": [
    {
      "claim":            "Infosys margin declined 0.8pp in FY2024",
      "citation":         { "document": "Infosys_AR_2024.pdf", "page": 31,
"snippet": "..." },
      "audit_status":     "verified",
      "confidence":       0.91
    }
  ],
  "flagged_uncertain":    ["TCS capex guidance for FY2025"],
  "blocked_unverifiable": [],
  "agents_invoked":       ["PlannerAgent", "RetrieverAgent", "AnalystAgent",
...],
  "latency_ms":           3240
}
```

3. Retrieval Architecture

3.1 Why Retrieval Quality Is the Hard Problem

At 10M documents, retrieval quality matters more than the frontier model. At 50 documents, the same principle applies: a state-of-the-art LLM cannot recover from retrieving the wrong chunk. Every downstream agent — analyst, comparator, auditor — operates on what the retriever returns. Garbage in, garbage out with citations attached.

3.2 Hybrid Retrieval: BM25 + Dense

Retriever	Strengths
BM25 (keyword)	Exact figure matching ('20.7%'), ticker symbols, fiscal year references, proper nouns. Precision at scale.
Dense (bi-encoder)	Semantic similarity — 'revenue growth' matches 'top-line expansion'. Handles paraphrase and domain terminology.
Hybrid (RRF fusion)	Reciprocal Rank Fusion combines both ranked lists. Better than weighted score combination in most empirical evaluations.

3.3 Reranking: Cross-Encoder

After hybrid retrieval returns a candidate pool, a cross-encoder reranker (ms-marco-MiniLM) rescores each chunk by jointly encoding the query and chunk together. This is more accurate than bi-encoder similarity but too slow for first-stage retrieval at scale — hence the two-stage design.

Bi-encoder vs Cross-encoder: the architectural tradeoff

A bi-encoder encodes query and document independently — fast enough for ANN search over millions of vectors. A cross-encoder encodes them together with full attention — far more accurate but O(n) at query time. You use bi-encoders to retrieve a candidate pool (top 20-50 chunks), then a cross-encoder to rerank that pool. This is the standard production pattern.

3.4 Source Confidence Scoring

After reranking, each chunk receives a composite confidence score. Section type detection uses heuristics over the extracted text and page metadata — income statement sections, auditor's reports, and notes to accounts score higher than MD&A narrative or the chairman's letter. This is domain-specific logic that generic RAG systems do not have.

4. Semantic Kernel Design

4.1 Why SK Is Load-Bearing

The deliberate choice of Semantic Kernel over LangChain is architectural, not cosmetic. SK's Stepwise Planner drives task decomposition dynamically — the plan changes with the query. Agents are SK plugins with native functions. The kernel manages context passing between function calls. If you removed SK and replaced it with direct Groq API calls and manual orchestration, you would need to re-implement the planner, the plugin system, and the context management. SK is not a convenience wrapper here — it is the orchestration layer.

4.2 SK Concepts Used

SK Concept	How Used in FinSight
Kernel	Singleton configured with Groq via OpenAI-compatible endpoint. All agents access it.
Native Plugin	RetrieverAgent, AnalystAgent, ComparatorAgent, AuditorAgent — functions with typed inputs/outputs.
Semantic Function	SynthesizerAgent — prompt template that assembles the final report from verified claims.
Stepwise Planner	PlannerAgent — takes user task, generates ordered execution plan of plugin function calls.
Kernel Arguments	Context passing between agent hops. Citations thread forward via KernelArguments.

4.3 LangChain vs SK — The Tradeoffs

Since LangGraph is familiar, here is the honest comparison:

Dimension	LangGraph	Semantic Kernel
Orchestration model	Graph-based — nodes and edges explicit	Planner-based — plan generated by LLM
Plugin system	Tools with schemas	Native functions registered to kernel
Context passing	State dict through graph	KernelArguments between functions
Control flow	Explicit graph topology	Planner decides order dynamically
Ceremony	Less — more Pythonic	More — explicit registration
Production maturity	High — battle-tested	Growing — Microsoft-backed
Best for	Deterministic workflows	Dynamic task decomposition

For FinSight, the dynamic task decomposition is the point. A user should be able to ask any arbitrary financial analysis question and get a plan. That is why SK's planner is the right choice here, and why it is load-bearing.

5. Six-Week Build Plan

Week 1 — Semantic Kernel Internals

Milestone
Can explain SK design philosophy out loud without reference. First plugin running against Groq.

1. Read SK Python docs start to finish. Take notes. Understand kernel, plugins, native functions, semantic functions, memory, planners.
2. Throwaway experiment 1: kernel + Groq + one native plugin with one function. Understand what the kernel does when it calls a function.
3. Throwaway experiment 2: semantic function. Understand native vs semantic — when to use each.
4. Throwaway experiment 3: chain two plugins together. Understand how SK passes context between functions via KernelArguments.
5. Throwaway experiment 4: SK memory. Store and retrieve. Understand the abstraction.
6. Throwaway experiment 5: Stepwise Planner. Give it a goal. Read the plan it generates. Understand what it is doing under the hood.
7. Write first DECISIONS.md entry: 'Why SK over LangChain for this project.' You know LangChain well enough to compare honestly.

Week 2 — Ingestion Pipeline

Milestone
10-K ingested. Chunks retrievable from Qdrant with page metadata and section type.

8. Set up repo structure. pyproject.toml, .env.example, docker-compose.yml. Qdrant running locally.
9. PDF parser using PyMuPDF. Extract text with page numbers and positional metadata. Understand bounding boxes — you need this for snippet extraction.
10. Chunker: sliding window with overlap. Research chunk size tradeoffs before deciding. Document your decision.
11. Metadata extraction: document name, company, fiscal year, page number, section type (heuristics over text content).
12. Deduplication: same document ingested twice should not create duplicate chunks. Design your dedup key.
13. Version tracking: what happens when a filing is re-ingested? Old chunks replaced or versioned? Decide and document.
14. Qdrant indexing: design your collection schema carefully. What goes in the vector, what in the payload.

15. Ingestion script: takes PDF path + doc_id, runs full pipeline. Ingest one real 10-K. Verify chunks in Qdrant.

Week 3 — Retrieval Layer

Milestone

Retrieval returning ranked, scored, source-aware chunks. Test harness showing contribution of each layer.

16. BM25 retrieval over Qdrant payload. Understand what BM25 is actually computing — read the original paper or a rigorous explanation.
17. Dense retrieval using sentence-transformers. Understand why you normalise embeddings before cosine similarity.
18. Hybrid fusion using Reciprocal Rank Fusion. Understand why RRF outperforms weighted score combination. Document the decision.
19. Cross-encoder reranker using ms-marco. Understand the architectural difference between bi-encoder and cross-encoder. This is interview material.
20. Source confidence model: five signals, composite score. Decide on combination method — weighted average vs multiplicative. Document.
21. Retrieval test harness: given a query, show raw BM25, raw dense, after RRF, after reranking, after confidence scoring. Keep this for your eval framework.

Week 4 — Agent Layer

Milestone

End-to-end task decomposition and analysis working. Citations threading through every agent hop.

Warning

This is the hardest week. SK's Stepwise Planner has sharp edges. Budget for it taking longer than expected. If this bleeds into Week 5, compress Week 6 polish instead.

22. SK kernel setup for FinSight. Decide on plugin structure — one plugin per agent, or plugins as capabilities that agents use. Document the decision.
23. PlannerAgent using SK Stepwise Planner. Understand what the planner is doing — it calls the LLM to generate a function call sequence. Read the SK planner source.
24. RetrieverAgent as SK native plugin. Takes subtask string, calls retrieval layer, returns ranked chunks with citations. Understand how to pass rich objects through SK function calling.
25. AnalystAgent: takes retrieved chunks, extracts KPIs and figures, returns structured data. Every extracted fact carries its citation forward. Design your output schema carefully.

26. ComparatorAgent: receives results from multiple subtasks, synthesises cross-document comparisons with multi-source citations. Define what 'anomaly' means — statistical, rule-based, or LLM-judged.
27. Wire agents together through SK. Planner generates plan. Plan executes through agent chain. Verify citations thread through every hop without being dropped.

Week 5 — Compliance Layer

Milestone
System is functionally complete and compliance-grade. Every output is verified before release.

28. AuditorAgent: for every claim — does it have a citation? Does the snippet support the claim? Are there contradicting passages? Use LLM entailment checking.
29. Implement the three audit statuses: VERIFIED, UNCERTAIN, UNVERIFIABLE. Define exact trigger conditions for each. Document them.
30. Hallucination fallback: if composite confidence across all chunks for a subtask drops below threshold — do not generate. Return 'insufficient evidence found.' Hard cutoff.
31. SynthesizerAgent: assembles final structured report. VERIFIED claims included unmarked. UNCERTAIN claims included with explicit flags. UNVERIFIABLE claims omitted entirely.
32. Audit log: every analysis run writes a JSON artifact. Store in audit_logs/. Human-readable and machine-parseable.
33. Adversarial testing of compliance layer: ask questions where the answer is not in the documents. Ask leading questions designed to elicit hallucination. Verify the system blocks or flags every time.

Week 6 — Eval, Observability, Polish

Milestone
Adversarial tests passing. Fully documented. One-command Docker. Eval numbers in README.

34. Eval framework: retrieval recall (are the right chunks retrieved?), citation accuracy (does snippet support claim?), hallucination rate (what % of claims are flagged or blocked?). Build a test harness that runs a fixed query set and reports these metrics.
35. Adversarial query sets: questions whose answers are not in the documents; questions designed to elicit cross-document confusion; numerically precise questions where hallucination is obvious.
36. OpenTelemetry tracing: every agent invocation, retrieval call, and reranker call gets a span. Trace the full path of a request end to end.
37. ARCHITECTURE.md: full system design, every component, every design decision with reasoning.

38. DECISIONS.md: every non-obvious decision made, in the format: context / options considered / decision / reasoning / tradeoffs accepted.
39. Docker: one command to start everything. A stranger on a fresh machine should be running within 5 minutes.
40. README: clear, professional, with actual eval numbers. Show a real example input and output with citations.

6. Repository Structure

```
finsight/
├── agents/
│   ├── router.py          # PlannerAgent — SK Stepwise Planner
│   ├── retriever.py       # RetrieverAgent — SK native plugin
│   ├── analyst.py         # AnalystAgent — KPI extraction
│   ├── comparator.py      # ComparatorAgent — cross-doc synthesis
│   ├── auditor.py         # AuditorAgent — claim verification
│   └── synthesizer.py     # SynthesizerAgent — report assembly
├── retrieval/
│   ├── qdrant_store.py    # Collection management + vector search
│   ├── embedder.py        # Local sentence-transformer embeddings
│   ├── bm25.py            # BM25 over payload
│   ├── hybrid.py          # RRF fusion
│   ├── reranker.py        # Cross-encoder ms-marco
│   └── confidence.py      # Source confidence model (5 signals)
├── ingestion/
│   ├── parser.py          # PDF + DOCX → text with page metadata
│   ├── chunker.py         # Sliding window chunker
│   ├── metadata.py        # Section type detection heuristics
│   └── pipeline.py        # Orchestrates full ingest flow
├── api/
│   ├── main.py            # FastAPI app, lifespan, middleware
│   ├── routes/
│   │   ├── ingest.py      # POST /ingest
│   │   ├── query.py       # POST /query
│   │   └── eval.py        # GET  /eval
│   └── middleware/
│       └── guardrails.py  # Prompt injection detection
├── evaluation/
│   ├── harness.py         # Test runner
│   ├── queries/           # Adversarial + happy-path query sets
│   └── results/           # Eval run outputs
├── observability/
│   └── tracer.py          # OTel setup + @traced decorator
├── core/
│   ├── sk_kernel.py       # SK kernel singleton
│   ├── groq_client.py     # Groq async client wrapper
│   └── config.py          # pydantic-settings
├── data/
│   └── filings/           # Seed PDFs: Infosys, TCS, Wipro FY22-24
├── audit_logs/            # Per-run JSON artifacts
├── docs/
│   ├── ARCHITECTURE.md
│   └── DECISIONS.md
├── tests/
├── docker-compose.yml
├── pyproject.toml
├── .env.example
└── README.md
```

7. Seed Data

Do not use synthetic or random PDFs. The system should be demonstrated on real data with real inconsistencies — that is what stress-tests it honestly and makes the portfolio piece credible.

Company	Filings	Source
Infosys	Annual Report FY2022, FY2023, FY2024	infosys.com/investors
TCS	Annual Report FY2022, FY2023, FY2024	tcs.com/investors
Wipro	Annual Report FY2022, FY2023, FY2024	wipro.com/investors

9 filings minimum. All publicly available. Indian IT sector is ideal — comparable business models, consistent fiscal year (April-March), and enough variation in margin trajectories to make cross-document comparison non-trivial.

Sample analysis tasks to test against

1. Compare Infosys and TCS operating margins FY2022-2024 and flag any anomalies or risk disclosures related to margin pressure. 2. What were the key revenue drivers for Wipro in FY2024 and how did they differ from FY2022? 3. Identify all mentions of attrition risk across all three companies in FY2023 and summarise the mitigations disclosed. 4. What guidance did each company provide for FY2025 capex and how does it compare to actual FY2024 capex?

8. Documentation Standards

8.1 DECISIONS.md Format

Every non-obvious design decision gets logged immediately, not at the end. This document is as valuable as the code in an interview.

```
## [Date] — Why X over Y

**Context:** what problem were you solving
**Options considered:** what you evaluated
**Decision:** what you chose
**Reasoning:** why
**Tradeoffs accepted:** what you gave up
```

8.2 Commit Standards

- Commit daily with meaningful commit messages
- Format: [component] action — e.g. [retrieval] add cross-encoder reranker with ms-marco
- Never commit broken code to main — use feature branches
- Every decision in DECISIONS.md should correspond to a commit where that decision was implemented

8.3 README Standards

- Clear one-liner at the top — what it does, for whom, why it matters
- Actual eval numbers — hallucination rate, retrieval recall, latency p50/p95
- Real example: input query → output report with citations
- Architecture diagram (even ASCII is fine)
- One-command quickstart that actually works on a fresh machine
- Link to ARCHITECTURE.md and DECISIONS.md

9. Interview Talking Points

9.1 The Resume Story in One Sentence

Elevator Pitch

I built a multi-agent financial analysis system where every claim is traceable to a verbatim source passage, uncertain claims are flagged rather than hallucinated, and every analysis generates a machine-readable audit log — because in compliance-heavy environments, an unverifiable AI output is worse than no output.

9.2 Questions You Should Be Able to Answer

- Why Semantic Kernel over LangChain? What does the Stepwise Planner actually do under the hood?
- Why RRF over weighted score combination for hybrid retrieval fusion?
- Why a cross-encoder for reranking instead of just using vector similarity scores?
- What is the difference between a bi-encoder and a cross-encoder architecturally?
- Why is the AuditorAgent a separate agent rather than a prompt instruction in the SynthesizerAgent?
- How does your confidence model handle the case where a figure appears in both audited statements and MD&A commentary with different values?
- What does your hallucination fallback actually trigger on? What is the threshold and how did you decide it?
- If you had to scale this to 10M documents, what would change first and why?
- What did your adversarial eval reveal? What is the system's actual hallucination rate?

9.3 The Design Decision That Gets the Most Discussion

The AuditorAgent as a structural verification pass rather than a prompt-level constraint. Be ready to explain: why prompt instructions are insufficient in a compliance context, what entailment checking actually does, and why the three-status model (verified / uncertain / unverifiable) is more useful than a binary pass/fail.

9.4 Connecting to EY's Context

At EY, the real barrier to GenAI adoption is not capability — it is trust. An auditor cannot cite an AI system in a client deliverable if they cannot trace every claim back to a primary source. FinSight is designed around exactly that constraint. The AuditorAgent, the citation threading, the audit log — these are not features added for portfolio value. They are the answer to the question every compliance-focused organisation is actually asking.

Build it. Document every decision.

The project gets better through iteration, not planning.